

Đại Học Công Nghiệp Thành Phố Hồ Chí Minh

Môn học : Thực Hành TK Vi Mạch Số với HDL

BÀI 6: Mạch cộng, nhân nhị phân

BÀI 4: MẠCH ĐẾM

1

Mạch cộng bán phần, toàn phần

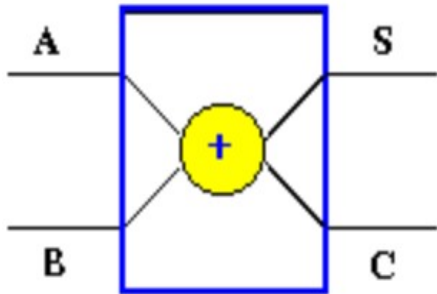
2

Mạch cộng nhị phân 4 bit

3

Mạch nhân nhị phân 4 bit

1. Mạch cộng bán phần (HA)



Chức năng: HA cộng hai số nhị phân A và B một bit không có nhớ

Biểu thức ngõ ra:

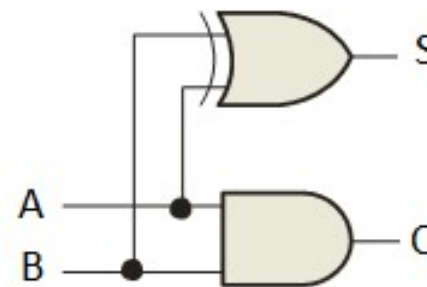
$$S = A \oplus B$$

$$C = A . B$$

Bảng trạng thái:

Vào		Ra	
A	B	S	C _o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sơ đồ mạch HA:



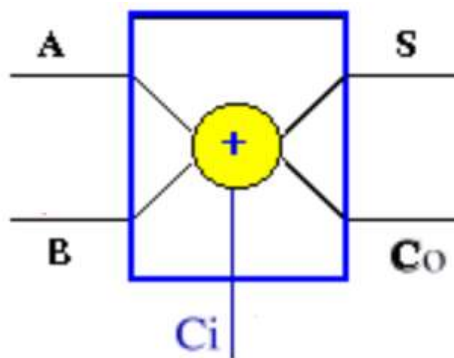
2. Mạch cộng toàn phần FA

Chức năng:

FA cộng hai số nhị phân A và B một bit có nhớ.

Bảng trạng thái:

Vào			Ra	
A	B	C_i	S	C_o
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

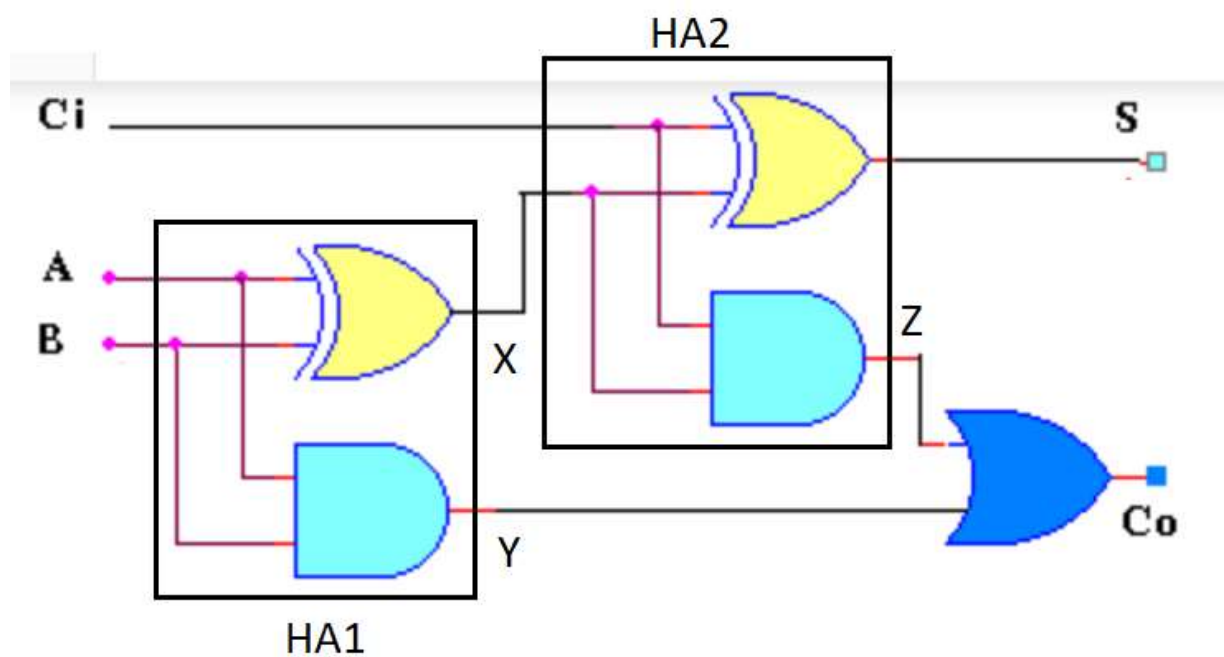


Biểu thức ngõ ra:

$$S = (A \oplus B) \oplus C_i$$

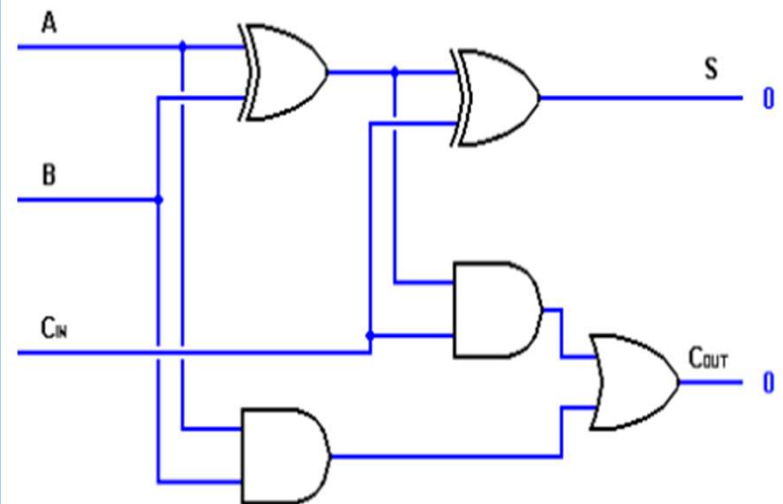
$$C_o = AB + (A \oplus B)C_i$$

Sơ đồ mạch logic mạch cộng FA



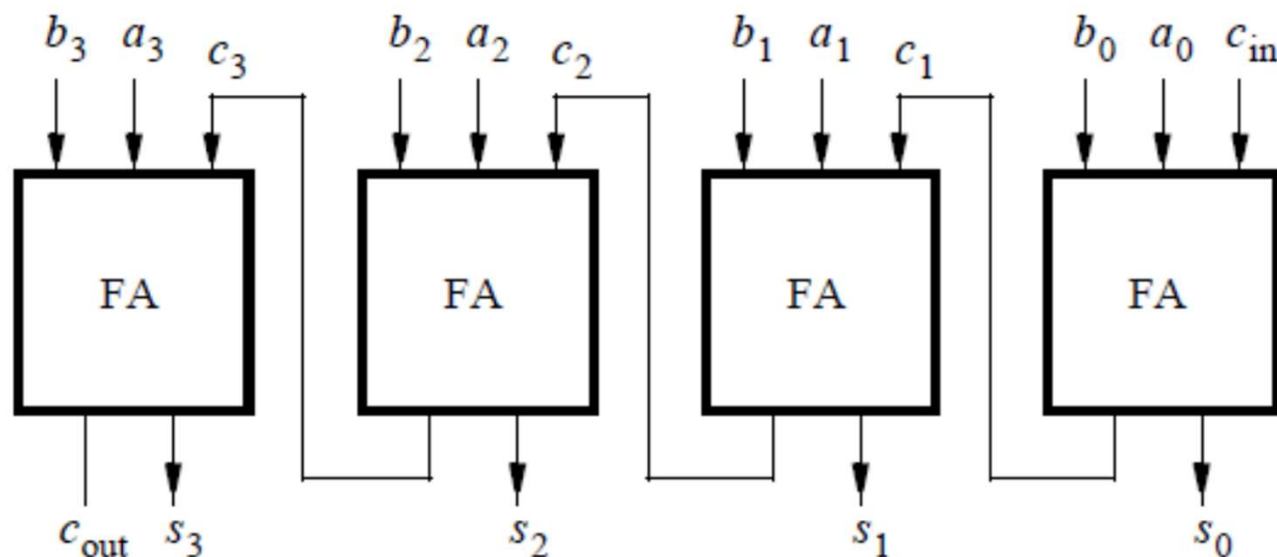
Code mô tả mạch cộng FA

```
module fa(co,s,a,b,ci);  
input a,b,ci;  
output co,s;  
wire n1,n2,n3,n4;  
xor (n1,a,b);  
xor (s,ci,n1);  
not (n2,n1);  
and (n3,n2,b);  
and (n4,ci,n1);  
or (co,n3,n4);  
endmodule
```



2. Mạch cộng nhị phân 4 bit

Thực hiện cộng 2 số nhị phân 4 bit A và B ta dùng FA ta có sơ đồ mạch sau:



Gán chân:
A = SW[3:0]
B = SW[7:4];
S = LEDG

- Xác định số ngõ vào, ngõ ra

Mạch có 4 ngõ vào cho số nhị phân 4 bit thứ nhất a tương ứng với các switch từ 0 tới 3. Có 4 ngõ vào cho số nhị phân 4 bit thứ hai b tương ứng với các switch từ 4 tới 7. Có 4 ngõ ra s tương ứng với các led green (LEDG) từ 0 tới 3 và bit nhớ c(out) tương ứng với led green 4.

- Các module trong chương trình:

Chương trình gồm 1 module chính cong4bit(SW, LEDG) và 1 module con fa(co,s,a,b,ci). Trong module chính, khai báo các ngõ vào, ngõ ra và thực hiện gọi module con 4 lần. Module con thực hiện mạch cộng toàn phần 1 bit theo sơ đồ logic như hình vẽ.

```
module cong4bit(SW, LEDG);  
input [7:0]SW;  
output [4:0]LEDG;  
wire [3:0]a,b;  
wire [4:0]s;  
wire n1;  
assign a=SW[3:0];  
assign b=SW[7:4];  
assign LEDG=s;  
fa (n1,s[0],a[0],b[0],1'b0);  
fa (n2,s[1],a[1],b[1],n1);  
fa (n3,s[2],a[2],b[2],n2);  
fa (s[4],s[3],a[3],b[3],n3);  
endmodule
```

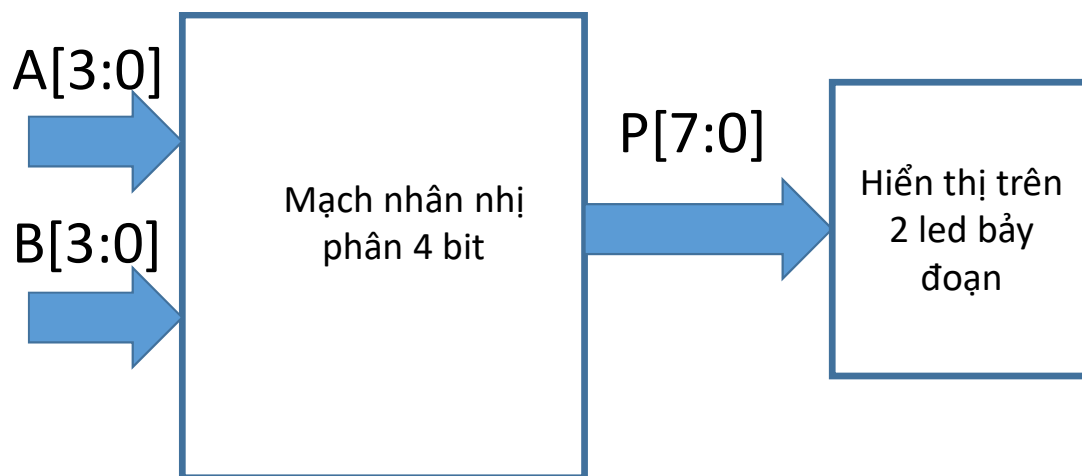
```
module fa(co,s,a,b,ci);  
input a,b,ci;  
output co,s;  
wire n1,n2,n3,n4;  
xor (n1,a,b);  
xor (s,ci,n1);  
not (n2,n1);  
and (n3,n2,b);  
and (n4,ci,n1);  
or (co,n3,n4);  
endmodule
```

2. Mạch nhân nhị phân 4 bit

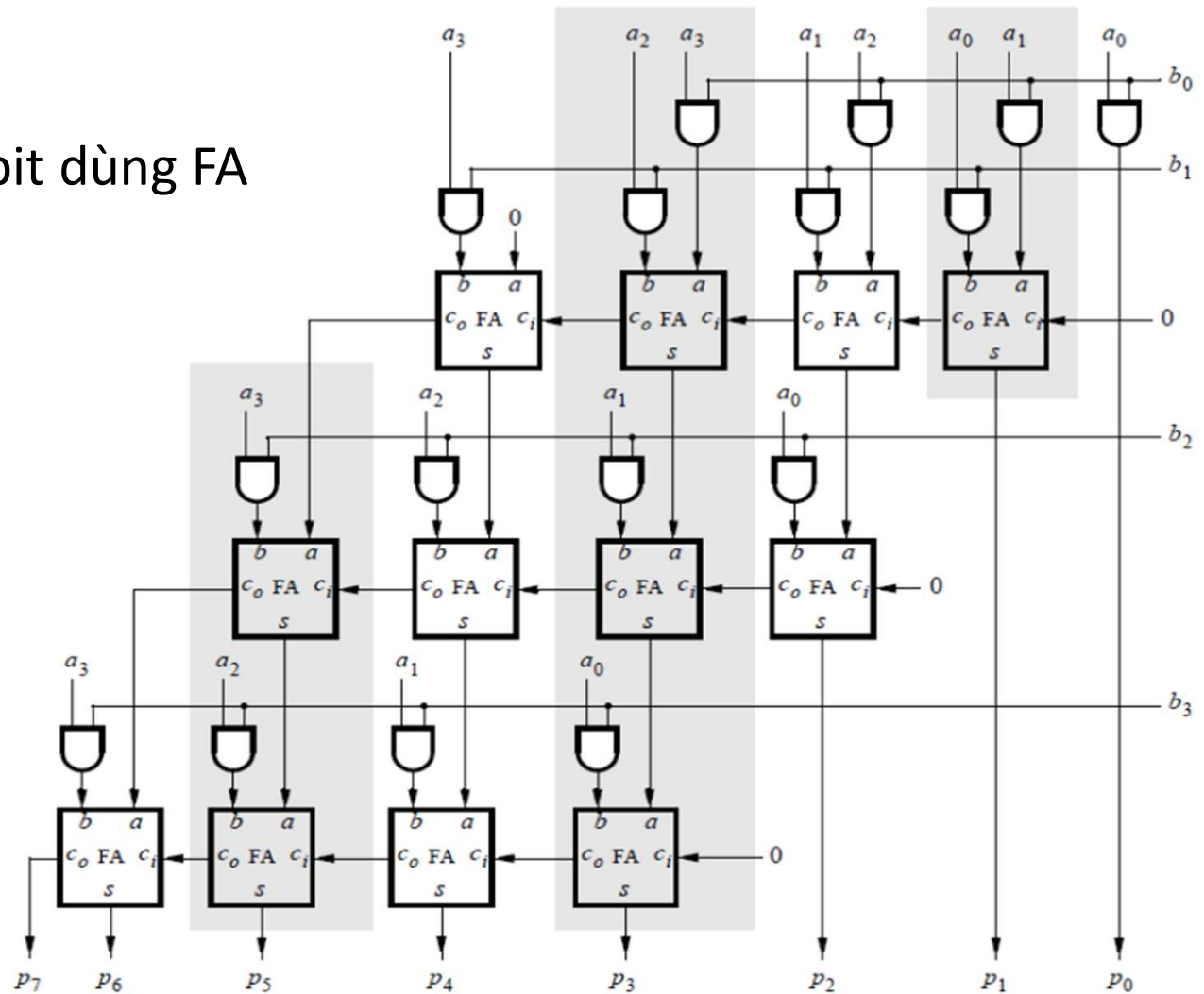
Thực hiện nhân 2 số nhị phân 4 bit A và B ta có kết quả sau:

				a_3	a_2	a_1	a_0
				b_3	b_2	b_1	b_0
				a_3b_0	a_2b_0	a_1b_0	a_0b_0
			a_3b_1	a_2b_1	a_1b_1	a_0b_1	
		a_3b_2	a_2b_2	a_1b_2	a_0b_2		
	a_3b_3	a_2b_3	a_1b_3	a_0b_3			
p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

VD: Sử dụng các SW11-8 để thực hiện cung cấp các giá trị cho số A, các SW3-0 để thực hiện cung cấp các giá trị cho số B. Các giá trị hexa của A và B hiển thị trên các LED 7 đoạn HEX6 và HEX4. Kết quả $P = A \times B$ hiển thị giá trị thập lục phân trên HEX1 và HEX0.



Sơ đồ mạch nhân 4 bit dùng FA



```
module machnhan (SW, HEX7, HEX6, HEX5, HEX4, HEX3, HEX2,
HEX1, HEX0);
input [15:0] SW;
output [0:6] HEX7, HEX6, HEX5, HEX4, HEX3, HEX2, HEX1, HEX0;
    wire [3:0] A, B;
    wire [7:0] P;
    wire [3:1] C_b1;
    wire [5:2] PP1;
    wire [3:1] C_b2;
    wire [6:3] PP2;
    wire [3:1] C_b3;
    assign A = SW[11:8];
    assign B = SW[3:0];
    assign P[0] = A[0] & B[0];
// module fa (a, b, ci, s, co);
fa b1_a0 (A[1] & B[0], A[0] & B[1], 1'b0, P[1], C_b1[1]);
fa b1_a1 (A[2] & B[0], A[1] & B[1], C_b1[1], PP1[2], C_b1[2]);
fa b1_a2 (A[3] & B[0], A[2] & B[1], C_b1[2], PP1[3], C_b1[3]);
fa b1_a3 (1'b0, A[3] & B[1], C_b1[3], PP1[4], PP1[5]);
```

```
// module fa (a, b, ci, s, co);
fa b2_a0 (PP1[2], A[0] & B[2], 1'b0, P[2], C_b2[1]);
fa b2_a1 (PP1[3], A[1] & B[2], C_b2[1], PP2[3], C_b2[2]);
fa b2_a2 (PP1[4], A[2] & B[2], C_b2[2], PP2[4], C_b2[3]);
fa b2_a3 (PP1[5], A[3] & B[2], C_b2[3], PP2[5], PP2[6]);
```

```
// module fa (a, b, ci, s, co);
fa b3_a0 (PP2[3], A[0] & B[3], 1'b0, P[3], C_b3[1]);
fa b3_a1 (PP2[4], A[1] & B[3], C_b3[1], P[4], C_b3[2]);
fa b3_a2 (PP2[5], A[2] & B[3], C_b3[2], P[5], C_b3[3]);
fa b3_a3 (PP2[6], A[3] & B[3], C_b3[3], P[6], P[7]);
```

```
hex7seg digit_7 (4'h0, HEX7);
hex7seg digit_6 (A, HEX6);
```

```
hex7seg digit_5 (4'h0, HEX5);
hex7seg digit_4 (B, HEX4);
```

```
hex7seg digit_3 (4'h0, HEX3);
hex7seg digit_2 (4'h0, HEX2);
hex7seg digit_1 (P[7:4], HEX1);
hex7seg digit_0 (P[3:0], HEX0);
endmodule
module fa (a, b, ci, s, co);
    input a, b, ci;
    output s, co;

    wire a_xor_b;

    assign a_xor_b = a ^ b;
    assign s = a_xor_b ^ ci;
    assign co = (~a_xor_b & b) | (a_xor_b & ci);
Endmodule
module hex7seg (hex, display);
    input [3:0] hex;
    output [0:6] display;
    reg [0:6] display;
```

```
always @ (hex)
    case (hex)
        4'h0: display = 7'b0000001;
        4'h1: display = 7'b1001111;
        4'h2: display = 7'b0010010;
        4'h3: display = 7'b0000110;
        4'h4: display = 7'b1001100;
        4'h5: display = 7'b0100100;
        4'h6: display = 7'b1100000;
        4'h7: display = 7'b0001111;
        4'h8: display = 7'b0000000;
        4'h9: display = 7'b0001100;
        4'hA: display = 7'b0001000;
        4'hb: display = 7'b1100000;
        4'hC: display = 7'b0110001;
        4'hd: display = 7'b1000010;
        4'hE: display = 7'b0110000;
        4'hF: display = 7'b0111000;
    endcase
endmodule
```

Cảm ơn các bạn đã lắng nghe!

